Übungsblatt 3: Interrupts und Timer

Mikrocomputertechnik 3 · Zustände, Zeitraster und Kontext auf Papier

Musterlösung

Ziel und Arbeitsweise

Sie analysieren Trap-Zustände, Einsprungadressen, Timer und externe Quellen. Alle sechs Pflichtaufgaben sind ohne Simulator oder Laborplatine lösbar. Annahmen: ein Hart im Machine Mode, RV32, keine Delegation, zunächst kein Nesting. RV32 hat 32-Bit-Integerregister; ein Hart ist ein Hardware-Ausführungskontext.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht sind 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgaben 4–5 (15 min), Aufgabe 6 (15 min). Lösungsbesprechungen gehören zu den Vorlesungsphasen.

Geben Sie zu Rechnungen Einheit, Rechenweg und Interpretation an. Hexadezimale Werte beginnen mit `0x`. Alle Werte sind didaktische Annahmen, keine Messungen.

Aufgabe 1: Trap-Zustand und Freigaben (6 Punkte)

Vor einem Timer-Trap gilt: Hauptprogramm im Machine Mode, globale Freigabe `mstatus.MIE = 1`, Timerfreigabe `mie.MTIE = 1`. Die Hardware schreibt den Rohwert `0x80000007` nach `mcause`. MIE bedeutet Machine Interrupt Enable, MTIE Machine Timer Interrupt Enable.

1. Trennen Sie Interrupt-Bit und Cause-Code. Was bedeutet der Code? Was bedeutet stattdessen `0x00000002`? (2 P)
2. Beschreiben Sie MIE und MPIE unmittelbar nach dem Eintritt sowie nach einer korrekten Rückkehr mit `mret`. MPIE ist Machine Previous Interrupt Enable. Ändert `mret` dabei automatisch `mtimecmp`? (2 P)
3. Der Timer ist später pending, MTIE ist 0, MIE ist 1. Wird im Modell ein Timer-Trap genommen? Hält diese Maskierung `mtime` an? Begründen Sie. (2 P)

Musterlösung 1

1. Auf RV32 ist Bit 31 das Interrupt-Bit: $I = 1$. Nach Entfernen dieses Bits ist $c = 7$, Machine Timer Interrupt. `0x00000002` hat $I = 0$, Code 2, also eine Illegal-Instruction-Exception, einen synchronen Trap wegen eines ungültigen Befehls. (je 1 P)
2. Die Hardware kopiert das alte MIE nach MPIE und setzt MIE auf 0. Hier ist danach MPIE 1. Bei korrekter Rückkehr stellt `mret` MIE aus MPIE wieder auf 1; die Software muss ihren weiteren Kontext erhalten haben. `mret` ändert den Timervergleich `mtimecmp` nicht. (1 P für Status, 1 P für Compare)

3. Kein Timer-Trap: Im Modell werden Pending, MTIE und MIE gemeinsam benötigt. Die Zeitquelle `mtime` läuft dennoch weiter; Maskierung betrifft die Interruptnahme, nicht den Oszillator. (je 1 P)

Eine wartende Anforderung ist weder eine Freigabe noch ein Ereigniszähler.

Aufgabe 2: Vektoradressen und Geräte-ID (5 Punkte)

Die Basis ist `BASE = 0x80000100`. `mtvec` besteht aus BASE und MODE in den untersten zwei Bits. Die Basis erfüllt die Ausrichtungsanforderungen des Modells.

1. Welcher `mtvec`-Registerwert stellt Vectored, MODE 1, ein? (1 P)
2. Geben Sie die Einsprungadressen für Timer-Interrupt Cause 7, External-Interrupt Cause 11 und Exception-Code 2 an. Begründen Sie. (3 P)
3. Der PLIC hat zufällig eine externe Quelle mit Geräte-ID 7. Benutzt sie automatisch den Timer-Vektorslot? (1 P)

PLIC bedeutet Platform-Level Interrupt Controller. Ein Slot enthält einen ausführbaren Befehl; die Tabelle ist keine automatisch geladene Zeigertabelle.

Musterlösung 2

`mtvec = 0x80000101`: MODE eins wird zu der passend ausgerichteten Basis hinzugenommen, die Basis selbst bleibt `0x80000100`.

Bei Interrupts im Vectored-Modus ist der Einsprung $a = \mathrm{BASE} + 4c$:

$$a_7 = \texttt{0x80000100} + 28 = \texttt{0x8000011C},$$

$$a_{11} = \texttt{0x80000100} + 44 = \texttt{0x8000012C}.$$

Die Exception geht zur Basis `0x80000100`, nicht zu BASE plus acht. Im Direct-Modus würden alle drei zur Basis gehen.

Geräte-ID sieben bleibt eine externe PLIC-Quelle, Kategorie elf. Sie benutzt im Modell den External-Slot; Claim ermittelt danach ihre ID. Kategorie und Geräte-ID sind verschiedene Identifikationsebenen. Bewertung: je 1 P für Registerwert, drei begründete Ziele und ID-Abgrenzung.

Aufgabe 3: Timerfrequenz und RV32-Zugriffe (5 Punkte)

Timerfrequenz 10 MHz, Intervall 1 ms, `mtime` und `mtimecmp` jeweils 64 Bit. Für die Teilzugriffe gilt Little Endian und starke Gerätezugriffsordnung.

1. Wie viele Timer-Ticks benötigt das Intervall? Würde eine Prozessorfrequenz von 100 MHz daran etwas ändern? (1 P)
2. Der Zähler geht von `0x00000001_FFFFFFFF` zu `0x00000002_00000000`. Software liest HI 1 vor dem Übergang und LO 0 danach. Welcher falsche Gesamtwert entsteht? Wie erkennt HI–LO–HI den Übergang? (2 P)
3. Der alte Vergleich ist `0x00000001_FFFFFFF0`, der neue `0x00000002_00000010`. Die aktuelle Zeit sei `0x00000001_FFFFFF80`. Warum wäre „neues LO zuerst, dann neues HI“ ungünstig? Geben Sie die geeignete Dreischrittfolge an. (2 P)

HI und LO bezeichnen obere und untere 32-Bit-Hälfte.

Musterlösung 3

1. $\Delta = 10 \cdot 10^6 \cdot 10^{-3} = 10000$ Timer-Ticks. Die Prozessorfrequenz bestimmt nicht automatisch die separat festgelegte Timerfrequenz. (1 P)
2. Gemischte Hälften ergeben `0x00000001_00000000`, einen zu alten Wert. Das zweite HI ist 2 und unterscheidet sich vom ersten HI 1. Der Versuch wird verworfen; erst gleiche HI-Werte erlauben das Zusammensetzen. (je 1 P)
3. Neues LO zuerst erzeugte den Zwischenvergleich `0x00000001_00000010`. Er ist kleiner als die angenommene aktuelle Zeit, obwohl alter und neuer Sollwert beide in der Zukunft liegen. Das kann eine unnötige Anforderung erzeugen. (1 P)

Geeignete Reihenfolge: LO auf `0xFFFFFFFF`, neues HI 2, neues LO `0x00000010`. Die Zwischenwerte lauten `0x00000001_FFFFFFFF` und `0x00000002_FFFFFFFF`; beide liegen hier in der Zukunft. (1 P)

Die Reihenfolge setzt die genannten Plattformbedingungen voraus. Interruptsperren allein stoppt den fortschreitenden Hardwarezähler nicht.

Aufgabe 4: Sollraster und verspätete Behandlung (5 Punkte)

Timer 10 MHz, Periode $\Delta = 10000$ Timer-Ticks, nächster Solltermin $d = 100000$ Ticks. Im betrachteten Zeitraum gibt es keinen Zählerumlauf und keinen arithmetischen Überlauf.

1. Die Behandlung liest $n = 102000$ Ticks. Vergleichen Sie den nächsten Termin bei `now + 10000` und beim absoluten Sollraster. Geben Sie beide Zeiten in ms an. (2 P)
2. In einem getrennten Fall liest die Behandlung erst $n = 153000$ Ticks. Bestimmen Sie die Zahl fälliger Solltermine und den nächsten Zukunftstermin. (2 P)
3. Warum darf eine Variable „vergangene Millisekunden“ nicht einfach bei jedem Handler-Eintritt um eins steigen? (1 P)

Musterlösung 4

1. Bei 10000 Ticks pro ms ist $n = 10{,}2$ ms. Relativ entsteht 112000 Ticks beziehungsweise 11,2 ms. Das absolute Raster setzt den Termin auf 110000 Ticks, 11,0 ms. Die relative Strategie übernimmt die Verspätung in die Phase. (je 1 P)
2. Für $n = 153000$ gilt

$$k = \left\lfloor \frac{153000 - 100000}{10000} \right\rfloor + 1 = 6, \qquad d' = 100000 + 6 \cdot 10000 = 160000.$$

Sechs Solltermine, 10 bis 15 ms, sind fällig. Der neue Termin 16 ms liegt strikt in der Zukunft und bleibt auf dem Sollraster. (je 1 P) 3. Ein Pending-Bit zählt nicht alle versäumten Perioden. Zusätzlich kann nach einem Compare-Update ein weiterer Timer-Eintritt entstehen, bevor die Pending-Anzeige die Änderung widerspiegelt. Verstrichene Zeit folgt aus `mtime` und einem Startwert, nicht aus der Zahl der Aufrufe. (1 P)

Ob eine Anwendung sechs Aktionen nachholt oder Überlast meldet, ist eine eigene Politik; die Rechnung allein legt dies nicht fest.

Aufgabe 5: PLIC-Zustände nachvollziehen (6 Punkte)

Ein Machine-Mode-Kontext, alle Quellen freigegeben, alle anfangs pending:

ID	Quelle	Priorität
1	UART	3
2	Sensor	7
3	Taster	7

Threshold 5. UART ist eine serielle Schnittstelle. Die Hardware bevorzugt bei gleicher Priorität die kleinere ID. Ein Claim ist ein expliziter Übernahmezugriff und wird nicht vom Threshold gefiltert.

1. Besteht anfangs eine externe Benachrichtigung? Welche ID wird zuerst geclaimt? Warum nicht ID 3? (2 P)
2. Software führt insgesamt vier Claims aus, auch wenn zwischenzeitlich keine Benachrichtigung mehr besteht. Nach jeder Nichtnull-ID bearbeitet sie die Geräteursache vollständig und meldet Complete derselben ID. Es entstehen keine neuen Ereignisse. Füllen Sie die Tabelle. (2 P)

Claim	zurückgegebene ID	noch pending danach	Benachrichtigung nach Bearbeitung
1			
2			
3			
4			

3. Was ist bei ID 0 zu tun? Was kann bei einer Level-Quelle passieren, wenn Software nur Complete meldet und die Geräteursache nicht beseitigt? (2 P)

Musterlösung 5

Anfangs besteht eine Benachrichtigung: Priorität sieben ist strikt größer als Threshold fünf. Der Claim liefert ID zwei, weil zwei und drei gleiche Priorität haben und die kleinere ID gewinnt. (je 1 P)

Claim	ID	noch pending	Benachrichtigung nach Bearbeitung
1	2	1, 3	ja, durch 3
2	3	1	nein
3	1	keine	nein
4	0	keine	nein

Die ersten beiden Claims entfernen jeweils die höher priorisierte wartende Quelle. Der dritte **explizite** Claim kann ID eins liefern, obwohl deren Priorität drei keine Benachrichtigung bei Threshold fünf erlaubt. Die Gerätenbearbeitung und Completion verhindern neue Anforderungen im Modell. Bewertung: Claimfolge 1 P, Pending und Benachrichtigungen 1 P.

ID null bedeutet keine Quelle: kein Gerätehandler, kein Complete mit null. (1 P) Eine weiter aktive Level-Ursache kann nach Complete wieder vom Gateway angenommen werden; wiederholte Interrupts sind dann möglich. Claim allein quittiert das Gerät nicht. (1 P)

Aufgabe 6: Geteilter Zustand und Nesting (5 Punkte)

1. Ein vorzeichenloser 32-Bit-Millisekundenzähler hat `last = 0xFFFFFFF0` und `now = 0x00000020`. Berechnen Sie die Differenz modulo 2^{32} und nennen Sie ihre Grenze. (1 P)
2. Hauptprogramm liest Flag 1, eine ISR setzt es erneut auf 1, dann löscht das Hauptprogramm auf 0. Bleibt die zweite Markierung erhalten? Ändert `volatile` diesen Ablauf zu einer atomaren Gesamtaktion? (1 P)
3. Nur für ISR-Frames stehen 256 Byte bereit, je Frame werden im Modell 64 Byte benötigt. Wie viele Frames passen höchstens? (1 P)
4. Beim äußeren Trap wird `mepc = 0x80001000` gesetzt. Software gibt Interrupts wieder frei, bevor sie `mepc` sichert. Ein innerer Trap schreibt `0x80002040` nach `mepc`. Welche Rückkehrinformation fehlt, und was muss vor der Wiederfreigabe grundsätzlich geschehen? (2 P)

ISR bedeutet Interrupt Service Routine. Ein Frame ist ein Software-Sicherungsrahmen. Das Budget in Teil 3 schließt alle anderen Stackverwendungen bereits aus.

Musterlösung 6

1. Sechzehn Millisekunden bis zum Umlauf und zweiunddreißig danach ergeben 48 ms. Die Beobachtungen dürfen nicht einen ganzen unbemerkten Umlauf auseinanderliegen. (1 P)
2. Die zweite Markierung ist nach dem Löschen nicht unterscheidbar vorhanden. `volatile` macht die Lese-Entscheide-Schreibfolge nicht atomar. Bei Bedarf braucht die Übernahme eine geeignete kurze kritische Sektion oder Datenstruktur. (1 P)
3. $256/64 = 4$ Frames, unter genau den angegebenen Budgetannahmen. (1 P)
4. Die ursprüngliche Hauptprogramm-Fortsetzungsadresse `0x80001000` fehlt. Der innere Rücksprung kann funktionieren, der äußere hat ohne Sicherung aber nicht mehr seinen richtigen Wert. (1 P) Vor Wiederfreigabe müssen erforderliche allgemeine Register, Trap-Rückkehrzustand, Status und geänderte Masken passend gesichert sein. Die erlaubten Quellen, Tiefe und Stackanforderungen brauchen eine ausdrückliche Nesting-Politik. (1 P)

Die Kontextsicherung ist kein Ersatz für die notwendige Gerätebehandlung.

Optional: reine Rechnung oder konkrete Simulation

Die C-Funktion `next_future` aus den Folien kann auf dem Host für die Fälle `(100000, 99999, 10000)`, `(100000, 100000, 10000)` und `(100000, 153000, 10000)` geprüft werden. Erwartete Ergebnisse sind `100000`, `110000` und `160000`. Das prüft keine Geräteadresse.

Eine Renode-Untersuchung benötigt dagegen eine vorhandene, konkret benannte Plattform und passende Firmware samt Trap-Eintritt. Dokumentieren Sie Timerfrequenz, Freigaben und tatsächlich beobachtete Zustände. Simulatorzeit und Hostzeit sowie Modellprüfung und reale Chiplatenz bleiben verschiedene Nachweise.